

실전반 5회 · SOP와 Skill로 로컬 에이전트 구성

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — 환경 복구와 준비된 도구 입출력 확인 · 50분](#)
- [4. 활동 2 — SOP 절 검색과 적용 대상·버전 필터 연결 · 50분](#)
- [5. 활동 3 — SKILL.md의 판단 순서와 인용 규칙 수정 · 50분](#)
- [6. 활동 4 — LangChain 도구로 근거 있는 조치 제안 생성 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [강사 노트](#)
- [용어](#)

날짜·시간	2027-01-25 (월) 14:00~18:00	모듈	M3 · 원 일정 8번
블록	구현 B6 · B7 / 확장 B4	산출물	B4+B6→B7 근거 기반 제안
완료 확인	스킬 지침과 문서 근거를 구분하고 근거 없으면 보류		

회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

구현 새로 만든다 · **확장** 조건·연결을 바꾼다 · **플랫폼** 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (합수·질의·정책 수정과 검증)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 10/31	구현	구현	구현						
2회 11/07		확장	확장						
3회 11/21			확장	구현					
4회 11/28		확장	확장		구현			구현	
5회 01/25				확장		구현	구현		
6회 01/26			플랫폼	확장+P			확장		
7회 01/27						확장+P	확장+P	구현+P	
8회 01/28	확장				확장			확장	
9회 01/29									확장+P

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 11/14	구현	구현							
2회 01/25		확장	구현						
3회 01/26				구현					
4회 01/27		확장	확장		구현			구현	
5회 01/28				확장		구현			
6회 01/29							구현	구현	
7회 02/01			플랫폼	확장+P					
8회 02/02						확장+P	확장+P	확장+P	
9회 02/03									확장+P
10회 02/04	확장				확장			확장	확장

이번 회차가 다루는 블록

이번 회차에서 할 일

방학이 끝나고 처음 모이는 날이다. 첫 활동에서 11월 체크포인트대로 환경이 돌아왔는지 확인한다. 그다음 4회차에 만든 사건(`event_id`)을 받아 **근거 있는 조치 제안**을 만드는 Agents 계층의 B7 을 세운다. 에이전트는 네 도구(설비 맥락 · 최근 구간 · SOP 검색 · 제안 검사)를 부르고, 할당된 `SKILL.md` 의 판단 순서를 따른다. 여기서 세 가지를 섞지 않는 것이 핵심이다. **Skill** 은 **판단 순서와 중단 규칙**, **SOP 절** 은 **근거**, **도구**는 **실제 조회와 검사**다. 오늘은 SOP 검색 필터(설비 범위 · 유효 버전 · 그래프 확인)와 `SKILL.md` 의 틀린 규칙, 그리고 지어낸 인용을 막는 근거 검증 코드를 고친다. 회차가 끝나면 `HYD-01` 은 `REDUCE_LOAD`, `HYD-02` 는 `FAN_BOOST` 를 각자의 SOP 절을 인용해 제안하고, SOP 가 없는 `HYD-03` 과 검색되지 않은 인용은 `HOLD` 로 멈춰야 한다.

학습 목표

- 체크포인트의 시험 구간 해시·그래프 적재 수·서버 상태로 방학 전 환경이 복구되었는지 판정한다.
- SOP 절 검색의 세 겹 거르기(`asset_scope` 경계 · `status active` · `APPLIES_TO` 확인)를 고치고, 각 겹이 무엇을 걸러내는지 실제 검색 결과로 보인다.
- `SKILL.md` 의 도구 목록·판단 순서·인용 규칙·중단 규칙을 고치고, 문서 규칙과 코드 검사가 하는 일을 구분해 설명한다.
- 근거 검증이 (`doc_id`, `version`, `section`) 이 모두 같은 검색 결과만 인정하도록 고치고, 지어낸 인용이 `HOLD` 로 바뀌는 것을 확인한다.

1. 시작 전 준비 (복습·환경 확인)

방학 동안 컨테이너가 내려갔을 수 있다. 상태를 먼저 본다(`Exited` 면 `docker compose up -d`).

```
cd lecture/system
docker compose ps --format '{{.Name}} {{.Status}}'
curl -s localhost:8910/health; echo
```

```
hydops-neo4j Up About an hour
hydops-postgres Up About an hour
{"ok":true}
```

4회차 복습: 감지는 규칙과 이동 통계로 하고, 사건은 설비·유형당 열린 것 하나만 만든다. 오늘 에이전트는 **이미 만들어진 사건**을 입력으로 받는다. 에이전트가 센서 원시 배열을 읽고 고장을 판정하지 않는다.

2. 핵심 개념

Skill 은 업무 지침, SOP 는 근거, 도구는 실행 기능

SKILL.md 에 승인 규칙을 썼다는 것만으로 승인 검사가 구현되지는 않는다. 검사는 도구의 코드가 한다.

Skill · SKILL.md

무엇을 어떤 순서로 판단하나

- 센서 품질 확인 → 대상 설비 → SOP 검색 → 제안 → 승인 확인 → 실행 → 재측정
- 인용 규칙: 검색 결과에 실제로 나온 절만
- 중단 규칙: 근거 없으면 HOLD, 센서 오류면 SENSOR_CHECK
- 팔레(수업용 1.8절 자) · 에이전트가 실행 시 로드

SOP · 문서 근거

무엇이 허용되고 누가 승인하나

- 5종: 센서 오류 점검 · 냉각 이상 확인 · 승인 부하 감소 · 조치 후 재점검 · 미개선 이관
- 같은 양식: 적용 설비 · 발동 조건 · 확인할 관측 · 허용 조치 · 승인 주체 · 재점검 기준
- 문서 ID 버전 유효일 → 긴 본문은 검색 도구로 불러온다

도구 · 실행 기능

실제로 조회하고 실행하고 검사한다

- DB-그래프 조회, 시뮬레이터 조치, 재측정 편집
- 허용 조치-값 범위-승인 기록-사건 유효성-중복 키를 코드에서 확인
- LangChain 도구 · MCP 서버 · 플랫홈 프로세스가 같은 함수를 공유

도구	입력	결과와 제한	누가 호출하나
get_asset_context	asset_id	센서 · 적용 SOP · 허용 조치(값 범위)	에이전트
get_recent_window	asset_id, 기간	관측 요약 · 단위 · 품질 플러그 — 원시 배열은 주지 않는다	에이전트
search_sop	asset_id, event_type	문서 ID · 버전 · 절 · 근거 본문 (설비-active 필터 + 그래프 확인)	에이전트
propose_action	event_id, 근거 참조	제안과 허용 범위 검사 결과 — 실행하지 않는다	에이전트
execute_sim_action	event_id, 승인 ID, 조치값	승인 테이블 기록을 검사한 뒤 실행하고 결과 기록	오케스트레이션(승인 후)
verify_recovery	조치 ID, 재측정 기간	회복 · 미개선 · 데이터 부족	오케스트레이션

Skill · SOP · 도구의 역할

2.1 세 가지를 섞지 않는다

	답는 것	이 시스템의 파일	틀리면 생기는 일
Skill	무엇을 어떤 순서로 판단하나, 언제 멈추나	<code>skills/hydraulic-cooling-response/SKILL.md</code>	센서 오류인데 설비 조치를 제안, 검색 안 된 절 인용
SOP	무엇이 허용되고 누가 승인하나 (근거)	<code>hydops/b6_sop/docs/*.md</code> → Neo4j <code>SOPSection</code> 42절	다른 설비-폐기 버전의 절이 근거가 됨
도구	실제로 조회하고 검사한다	<code>hydops/b7_agent/tools.py</code> 4종	값 범위-승인 없이 진행

SKILL.md 에 "승인 없이 실행하지 않는다"고 적었다고 승인 검사가 생기지 않는다. 검사는 `propose_action` · `execute_sim_action` 코드가 한다. 반대로 코드에 검사가 있어도 Skill 순서가 틀리면 에이전트가 센서 품질을 보기 전에 조치를 고른다.

2.2 SOP 검색은 세 겹으로 거른다

`hydops/b6_sop/search.py` 의 `search_sop` 는 학습이 아니라 실행 시 근거를 가져오는 RAG 다.

```
# lecture/system/hydops/b6_sop/search.py (발췌)
flt: dict = {"asset_scope": {"$like": f"|{asset_id}|"}}
```

```

if not include_superseded:
    flt["status"] = {"$eq": "active"}
docs = vector_store().similarity_search_with_score(query, k=k * 3, filter=flt)

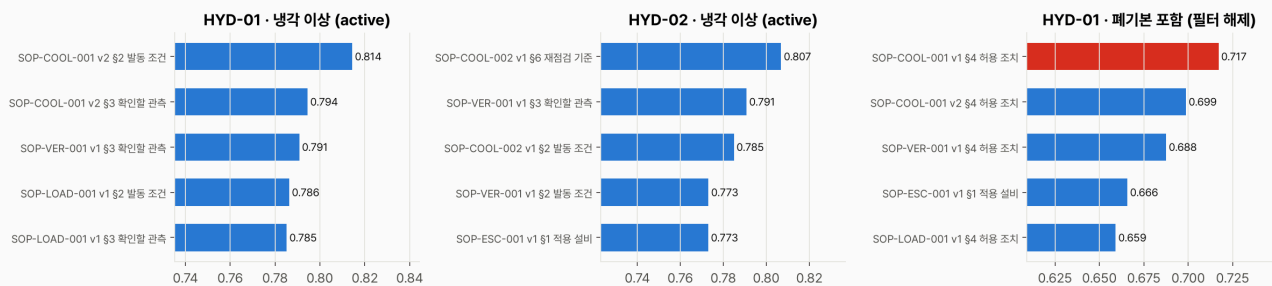
# 그래프 확인: 실제 APPLIES_TO 관계가 있고 사건 유형이 맞는 SOP 만 근거로 인정
applicable = {(s["doc_id"], s["version"]) for s in
               graph.applicable_sops(asset_id, event_type)}

...
for doc, score in docs:
    m = doc.metadata
    key = (m["doc_id"], m["version"])
    if key not in applicable or m["id"] in seen:
        continue

```

검	조건	걸러내는 것
① 설비 범위	<code>asset_scope</code> 가 `	HYD-01
② 유효 버전	<code>status = 'active'</code>	폐기된 SOP-COOL-001 v1
③ 그래 프 확인	<code>(SOP)-[:APPLIES_TO]->(Asset)</code> 이 있 고 <code>event_type</code> 일치	같은 설비 범위지만 사건 유형이 다른 절 (냉 각 이상 검색에 SOP-SEN-001)

B6 · Neo4jVector SOP 검색 — 설비·버전 필터가 근거의 범위를 정한다 (빨강 = 폐기된 1판)



설비·버전 필터별 검색 결과

주의: 메타데이터 필터 ①②와 그래프 확인 ③은 겹치는 부분이 있다. ③만 있어도 v1 은 빠진다. 그
래도 ①②를 두는 이유는 벡터 후보 `k*3` 개를 다른 설비 절로 채우지 않기 위해서다. 후보가 남의
절로 가득 차면 ③이 다 버리고 결과가 비어 버린다.

3. 활동 1 — 환경 복구와 준비된 도구 입출력 확인 · 50분

목표

체크포인트와 현재 환경을 대조하고, 에이전트에 줄 도구 네 개의 입력과 출력을 직접 호출해 확인한다.

따라 하기

1. 시험 구간 해시를 11월 체크포인트와 비교한다.

```
PYTHONPATH=. .venv/bin/python -c "  
import hashlib, json  
from hydops.b5_detect import evaluate as E  
_, base, test = E.fixed_split()  
print(len(base), len(test), hashlib.sha256(json.dumps([int(x) for x in  
    test]).encode()).hexdigest()[:16])  
"  
grep test_ids_sha256 ../checkpoints/2026-11-28_teamA/checkpoint.json
```

2. 4회차 실습이 그대로 통과하는지 본다.

```
PYTHONPATH=. .venv/bin/python -m pytest ../labs/practical/S04 -q
```

3. 그래프 적재 수와 에이전트 모드를 확인한다.

```
PYTHONPATH=. .venv/bin/python -c "  
from hydops.b4_ontology import graph; print(graph.graph_stats())  
from hydops.config import SETTINGS; print(SETTINGS.agent_mode,  
    SETTINGS.llm_model)  
from hydops.b7_agent import tools as T  
print([(t.name, list(t.args)) for t in T.build_tools()])  
"
```

4. 도구를 하나씩 호출한다. `build_tools()` 는 LangChain `@tool` 목록이고, 결과는 JSON 문자열이다.

```
PYTHONPATH=. .venv/bin/python -c "  
import json  
from hydops.b7_agent import tools as T  
tl = {t.name: t for t in T.build_tools()}  
print(json.loads(tl['get_recent_window'].invoke({'asset_id': 'HYD-02', 'seconds':  
    60}))['sensors']['TS1'])
```

```

r = json.loads(tl['search_sop'].invoke({'asset_id': 'HYD-02', 'event_type':
    'COOLING_ANOMALY', 'query': '허용 조치'}))
print(r['filter'], [(h['doc_id'], h['version'], h['section'], h['heading']) for h
    in r['hits']])
for args in [{ 'event_id': 'EVT-20260914-131041-HYD-01-f81e', 'action_id':
    'REDUCE_LOAD', 'value': 0.4, 'citations': [{ 'doc_id': 'SOP-COOL-001',
    'version': 2, 'section': '4' } ] },
    { 'event_id': 'EVT-20260914-130529-HYD-03-3bf1', 'action_id':
    'REDUCE_LOAD', 'value': 0.8, 'citations': [] } ]:
    r = json.loads(tl['propose_action'].invoke(args)); print(r['ok'],
        [(c['check'], c['ok'], c['detail']) for c in r['checks']])

```

`event_id` 는 수업 DB 의 사건으로 바꾼다(`curl -s "localhost:8800/api/events?all_runs=true"`). `propose_action` 은 **검사만** 하고 실행하지 않으므로 닫힌 사건에 불려도 기록이 바뀌지 않는다.

확인 포인트

1단계: `244 180 2bb1593b85bc83a9` 와 체크포인트의 값이 같다. 2단계: `9 passed` .

3단계 (Event 수는 DB 마다 다르다):

```

{'nodes': {'Asset': 3, 'Resource': 3, 'Sensor': 9, 'Action': 4, 'Driver': 2,
    'Process': 2, 'KPI': 2, 'SOP': 7, 'SOPSection': 42, 'Event': 6}, 'relationships':
    {'HAS_COMPONENT': 3, 'HAS_SENSOR': 9, 'AFFECTS': 6, 'RUNS_ON': 6, 'MEASURED_FOR':
    6, 'APPLIES_TO': 10, 'ALLOWS': 6, 'HAS_SECTION': 42, 'SUPERSEDES': 1, 'ON_ASSET':
    6, 'CITES': 15}}
llm gpt-4.1-mini
[('get_asset_context', ['asset_id']), ('get_recent_window', ['asset_id',
    'seconds']), ('search_sop', ['asset_id', 'event_type', 'query']),
    ('propose_action', ['event_id', 'action_id', 'value', 'citations'])]

```

`SOP 7 · SOPSection 42 · APPLIES_TO 10 · SUPERSEDES 1` 이 3회차 적재와 같으면 그 래프는 복구된 것이다. `agent_mode` 는 OpenAI 키가 있으면 `llm` , 없으면 `offline` 이다.

4단계:

```

{'unit': '°C', 'samples': 60, 'quality': {'n': 60, 'valid': 60, 'valid_ratio':
    1.0, 'flags': {'OK': 60}, 'longest_gap_s': 0}, 'valid_mean': 44.96, 'valid_min':
    44.4, 'valid_max': 45.62, 'last_valid': 44.97, ...,
    'longest_valid_above_alarm_s': 0, 'sustained_anomaly': False, 'sensor_state':
    'VALID'}
{'asset_scope': {'$like': '|HYD-02|'}, 'status': {'$eq': 'active'}} [ ('SOP-VER-
    001', 1, '4', '허용 조치'), ('SOP-COOL-002', 1, '4', '허용 조치'), ('SOP-ESC-001', 1,
    '4', '허용 조치'), ('SOP-ESC-001', 1, '1', '적용 설비'), ('SOP-VER-001', 1, '1', '적용

```

```

설비']]
False [('event_exists', True, ...), ('event_open', False, 'CLOSED'),
('action_allowed_by_sop', True, 'REDUCE_LOAD for HYD-01'), ('value_in_range',
False, '0.4 in [0.6, 1.0]'), ('has_citation', True, '1 citations'),
('not_already_executed', False, '1 prior executions')]
False [('event_exists', True, ...), ('event_open', False, 'HOLD_NO_EVIDENCE'),
('action_allowed_by_sop', False, 'REDUCE_LOAD for HYD-03'), ('has_citation',
False, '0 citations'), ('not_already_executed', True, '0 prior executions')]

```

- `get_recent_window` 는 원시 배열 대신 코드가 계산한 요약(유효 비율 · 지속 초 · `sensor_state`)만 준다.
- `propose_action` 은 검사 목록을 모두 돌려준다. 값 0.4 는 허용 범위 [0.6, 1.0] 밖이고, HYD-03 에는 REDUCE_LOAD 를 허용하는 SOP 가 없다.

흔한 실수

- 해시가 다른데 그대로 진행한다. 누군가 `evaluate.py` 나 축약본을 바꾼 것이다. 강사에게 알리고 체크포인트 코드로 되돌린다.
- 환경 확인을 위해 `pytest tests -q` 를 실행한다. 시스템 테스트는 사건 테이블을 비우는 픽스처가 있어 **공유 DB 에서 실행하지 않는다**. 확인은 lab 테스트와 읽기 조회로 한다.
- `search_sop` 결과가 HYD-02 인데 SOP-COOL-002 §4 가 첫 줄이 아니라고 필터 오류로 판단한다. 순위는 임베딩 점수다. 확인할 것은 **다른 설비의 절이 없는가**다.

4. 활동 2 — SOP 절 검색과 적용 대상·버전 필터 연결 · 50분

목표

`labs/practical/S05/search_lab.py` 의 `build_filter` 와 `confirm_with_graph` 를 고친다. 검색 조립(`search_sop_lab`)은 제공되며, 기본 검색기는 시스템과 같은 Neo4jVector 다.

따라 하기

1. 시작본 두 함수를 읽는다.

```

# lecture/labs/practical/S05/search_lab.py (시작본 발췌)
def build_filter(asset_id, include_superseded=False):
    flt: dict = {"asset_scope": {"$like": asset_id}}
    return flt

```

```
def confirm_with_graph(candidates, asset_id, event_type, k=4,
                       include_superseded=False):
    hits = []
    for meta, text, score in candidates:
        hits.append({...})
        if len(hits) >= k:
            break
    return hits
```

2. 시작본으로 HYD-01(k=4)과 HYD-02(k=8)를 검색한다.

```
PYTHONPATH=../../labs/practical/S05 .venv/bin/python -c "
import search_lab as S
for a, k in (('HYD-01', 4), ('HYD-02', 8)):
    r = S.search_sop_lab(a, 'COOLING_ANOMALY', k=k)
    print(a, r['filter'], [f"{h['doc_id']} v{h['version']} ${h['section']}" for
                          h in r['hits']])
" 2>&1 | grep -v Warn
```

3. `build_filter` 에 경계 문자 `|` 와 `status` 조건을, `confirm_with_graph` 에 `applicable_keys(...)` 확인과 같은 절 중복 제거를 넣는다. 같은 명령을 다시 실행하고, 시스템 `search_sop(a, 'COOLING_ANOMALY', k=8)` 결과와 비교한다.

확인 포인트

시작본:

```
HYD-01 {'asset_scope': {'$like': 'HYD-01'}} ['SOP-COOL-001 v2 $2', 'SOP-COOL-001
v2 $3', 'SOP-VER-001 v1 $3', 'SOP-COOL-001 v1 $2']
HYD-02 {'asset_scope': {'$like': 'HYD-02'}} ['SOP-COOL-002 v1 $6', 'SOP-VER-001
v1 $3', 'SOP-COOL-002 v1 $2', 'SOP-VER-001 v1 $2', 'SOP-ESC-001 v1 $1', 'SOP-ESC-
001 v1 $3', 'SOP-SEN-001 v1 $6', 'SOP-SEN-001 v1 $2']
```

- HYD-01 네 번째 근거가 폐기된 **SOP-COOL-001 v1 \$2** 다(겹 ② 없음).
- HYD-02 에 **센서 오류 SOP(SOP-SEN-001)** 절 두 개가 냉각 이상 근거로 들어왔다(겹 ③ 없음). SOP-SEN-001 의 `asset_scope` 는 `|HYD-01|HYD-02|` 라서 설비 필터만으로는 못 거른다.

고치기 뒤 (시스템 `search_sop` 결과와 같다):

```
HYD-01 {'asset_scope': {'$like': '|HYD-01|'}, 'status': {'$eq': 'active'}} ['SOP-
COOL-001 v2 $2', 'SOP-COOL-001 v2 $3', 'SOP-VER-001 v1 $3', 'SOP-LOAD-001 v1 $2']
```

```
HYD-02 {'asset_scope': {'$like': '|HYD-02|'}, 'status': {'$eq': 'active'}} ['SOP-COOL-002 v1 §6', 'SOP-VER-001 v1 §3', 'SOP-COOL-002 v1 §2', 'SOP-VER-001 v1 §2', 'SOP-ESC-001 v1 §1', 'SOP-ESC-001 v1 §3', 'SOP-COOL-002 v1 §4', 'SOP-ESC-001 v1 §2']
```

필터 없이 벡터 검색만 하면 HYD-02 질의에도 1위가 `SOP-COOL-001 v2 §2` (HYD-01 전용, 점수 0.814)였다. 검색 점수는 "비슷한 글"이지 "이 설비에 적용되는 글"이 아니다.

흔한 실수

- 경계 문자 없이 `$like: 'HYD-01'` 로 둔다. 지금 설비 셋에서는 결과가 같아 보이지만 `HYD-011` 이 생기면 섞인다. 테스트는 이 경우를 따로 검사한다.
- 그래프 확인을 `graph.applicable_sops(asset_id)` 로 부르며 사건 유형을 빠뜨린다. 기본값이 `COOLING_ANOMALY` 라 센서 오류 검색에서 SOP-SEN-001 이 모두 버려진다.
- `include_superseded=True` 인 감사용 검색에서 그래프 확인까지 active 로 묶는다. 이 경우 `APPLIES_TO` 는 있지만 폐기된 버전도 인정해야 한다(`applicable_keys` 의 두 번째 질의).

5. 활동 3 — SKILL.md의 판단 순서와 인용 규칙 수정 · 50분

목표

`labs/practical/S05/skills/hydraulic-cooling-response/SKILL.md` 의 틀린 지침 네 곳을 고치고, 스킬 로더가 시스템 프롬프트에 넣는 모양을 확인한다.

따라 하기

1. 시작본의 문제 부분을 읽는다.

```
tools: [get_asset_context, get_recent_window, search_sop, propose_action,
        execute_sim_action]

...
## 판단 순서
1. **SOP 검색** - `search_sop(asset_id, event_type)` 로 적용 SOP 의 절을 찾는다.
2. **제안** - `propose_action(...)` 으로 허용 범위를 검사한다.
3. **실행** - 검사가 통과하면 `execute_sim_action` 으로 조치를 실행한다.
4. **대상 설비 확인** - `get_asset_context(asset_id)` ...
5. **센서 품질 확인** - `get_recent_window(asset_id, 60)` ...
```

인용 규칙

- 냉각 이상 제안에는 항상 `SOP-COOL-001` §4 와 `SOP-LOAD-001` §4 를 인용한다.
- 이 Skill 문서도 판단 근거로 인용할 수 있다.

중단 규칙

- 적용 SOP 가 검색되지 않으면 `get\_asset\_context` 의 기본값으로 REDUCE_LOAD 를 제안한다.

2. 다음 기준으로 고친다.

절	고칠 내용	이유
frontmatter tools	execute_sim_action 제거	에이전트는 제안까지. 실행은 승인 기록 뒤 오케스트레이션이 한다
판단 순서	센서 품질(get_recent_window) → 대상 설비 (get_asset_context) → SOP 검색 → 제안 검사 → 승인 확인 → (실행은 부르지 않음) → 재측정	센서 오류면 설비 조치를 고르기 전에 멈춰야 한다
인용 규칙	절 번호 대신 "검색 결과 hits 에 실제로 나온 doc_id · version · section 만", 없으면 query 를 바꿔 재 검색, Skill 문서는 인용하지 않는다	절 번호를 박으면 검색되지 않은 절을 인용한다
중단 규칙	적용 SOP·검색 절이 없으면 HOLD , 검사 실패면 HOLD /이미 실행 이면 ESCALATE , SENSOR_FAULT 면 SOP-SEN-001 검색 후 SENSOR_CHECK	근거 없는 기본 조치 제안 금지

3. 로더로 프롬프트 길이를 확인한다.

```
PYTHONPATH=. .venv/bin/python -c "  
from pathlib import Path  
from hydops.b7_agent.skill_loader import build_system_prompt  
p = build_system_prompt(['hydraulic-cooling-response'],  
    Path('../labs/practical/S05/skills'))  
print(len(p)); print(p[:160])  
"
```

확인 포인트

- 시작본 프롬프트 길이 1203자, 고친 뒤(시스템 SKILL.md 와 같은 내용) 1746자. 로더는 본문을 최대 6,000자까지만 넣는다(MAX_CHARS).

- 프롬프트는 "너는 유압설비 운영 에이전트다. ..." 두 줄 뒤 **[할당된 스킬 가이드]** 제목 아래 SKILL.md 본문이 들어간다. 이 문서는 **지침**으로 들어갈 뿐, 코드 검사를 대신하지 않는다.
- 테스트는 판단 순서 절에서 네 도구 이름이 나오는 위치가 순서대로인지, 인용 규칙 절에 **\$숫자** 가 없는지, 중단 규칙 절에 **HOLD · SENSOR_CHECK** 가 있고 **REDUCE_LOAD** 가 없는지 본다.

흔한 실수

- 절 번호를 박아 넣는다. 실제 실행에서 "부하 감소 SOP §4 를 인용하라"고 적었더니 LLM 이 검색 되지 않은 절을 인용했고, 코드의 근거 검증이 HOLD 로 막았다(안전장치는 작동했지만 조치 제안은 실패). Skill 에는 "검색 결과에 있는 절만 인용"을 쓴다.
- HTML 주석으로 TODO 를 남겨 둔 채 제출한다. 로더는 주석까지 프롬프트에 넣는다.
- 판단 순서에서 실행 단계를 지우면서 "재측정" 단계까지 지운다. 7단계 "명령 성공만으로 종결을 제안하지 않는다"는 뒤 회차 판단에 필요하다.

6. 활동 4 — LangChain 도구로 근거 있는 조치 제안 생성 · 50분

목표

verify_citations_lab 을 고치고, 오프라인 에이전트 경로로 네 가지 결과(PROPOSE ×2, HOLD, SENSOR_CHECK)를 만든다. 실제 사건에 시스템 에이전트를 돌려 기록된 LLM 제안과 비교한다.

따라 하기

1. 시작본 근거 검증은 **doc_id** 만 비교한다. 지어낸 인용(폐기본 v1 §4)을 넣어 본다.

```
PYTHONPATH=../../labs/practical/S05 .venv/bin/python -c "
import search_lab as S
from hydops.b7_agent.agent import Proposal
trace = [{ 'tool': 'search_sop', 'result': S.search_sop_lab('HYD-01',
    'COOLING_ANOMALY')}]
p = Proposal(decision='PROPOSE', action_id='REDUCE_LOAD', value=0.8, citations=
    [{ 'doc_id': 'SOP-COOL-001', 'version': 1, 'section': '4'}],
    rationale='r')
f, problems = S.verify_citations_lab(p, trace); print(f.decision, problems,
    f.rationale)
" 2>&1 | grep -v Warn
```

2. 세 값 비교와 "인용 0개 → HOLD" 를 넣고 다시 실행한다.

3. 오프라인 에이전트 경로를 설비별로 돌린다. `offline_proposal` 은 시스템 `run_offline_agent` 와 같은 순서를 DB 사건 없이 따른다.

```
PYTHONPATH=../../labs/practical/S05 .venv/bin/python -c "  
import json, search_lab as S  
for a, st in (('HYD-01','VALID'), ('HYD-02','VALID'), ('HYD-03','VALID'), ('HYD-  
01','SENSOR_FAULT')):  
    o = S.offline_proposal(a, 'COOLING_ANOMALY', sensor_state=st)['final']  
    print(a, st, json.dumps({k: o[k] for k in  
        ('decision','action_id','value','citations')}, ensure_ascii=False))  
" 2>&1 | grep -v Warn
```

4. 실제 사건에 시스템 에이전트(오프라인 모드)를 돌린다. 도구 호출 순서와 결과를 본다.

```
PYTHONPATH=. .venv/bin/python -c "  
import json  
from hydops.b7_agent.agent import propose_for_event  
for eid in ['EVT-20260914-130529-HYD-03-3bf1', 'EVT-20260914-130352-HYD-01-2edd',  
    'EVT-20260914-131041-HYD-01-f81e']:  
    r = propose_for_event(eid, mode='offline')  
    print(eid[-11:], r['proposal']['decision'], [t['tool'] for t in  
        r['tool_trace']], r['proposal']['rationale'])  
" 2>&1 | grep -v Warn
```

확인 포인트

1단계, 시작본: `PROPOSE [] r` — 검색된 적 없는 v1 §4 가 통과했다. 고친 뒤:

```
HOLD ['SOP-C00L-001@v1#4'] 근거 검증 실패로 보류: 검색되지 않은 인용 ['SOP-C00L-001@v1#4']
```

3단계:

```
HYD-01 VALID {"decision": "PROPOSE", "action_id": "REDUCE_LOAD", "value": 0.8,  
"citations": [{"doc_id": "SOP-C00L-001", "version": 2, "section": "2"},  
{"doc_id": "SOP-C00L-001", "version": 2, "section": "3"}, {"doc_id": "SOP-VER-  
001", "version": 1, "section": "3"}]}  
HYD-02 VALID {"decision": "PROPOSE", "action_id": "FAN_BOOST", "value": 1.3,  
"citations": [{"doc_id": "SOP-C00L-002", "version": 1, "section": "6"},  
{"doc_id": "SOP-VER-001", "version": 1, "section": "3"}, {"doc_id": "SOP-C00L-  
002", "version": 1, "section": "2"}]}  
HYD-03 VALID {"decision": "HOLD", "action_id": null, "value": null, "citations":  
[]}  
HYD-01 SENSOR_FAULT {"decision": "SENSOR_CHECK", "action_id": "SENSOR_CHECK",
```

```
"value": null, "citations": [{"doc_id": "SOP-SEN-001", "version": 1, "section": "3"}, {"doc_id": "SOP-SEN-001", "version": 1, "section": "5"}]}
```

HYD-02 가 FAN_BOOST 인 것은 오프라인 에이전트가 허용 조치를 **그래프(get_asset_context)에 서 읽기** 때문이다. 조치 이름을 코드에 박았던 초기 버전은 REDUCE_LOAD 만 찾아 HYD-02 를 HOLD 했고, LLM 결과와 달랐다.

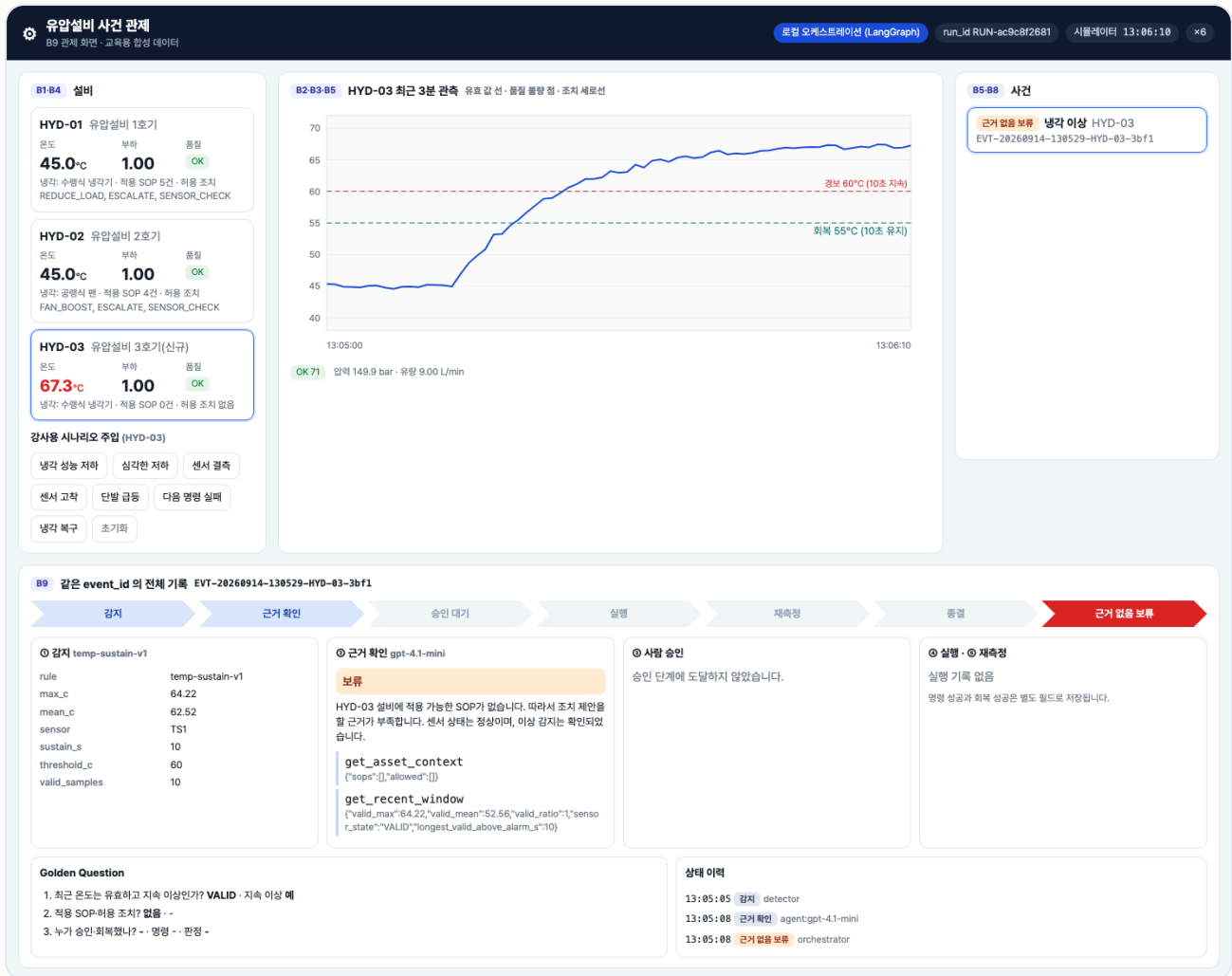
4단계:

HYD-03-3bf1 HOLD ['get_recent_window', 'get_asset_context', 'search_sop'] 이 설비에 적용되는 냉각 이상 SOP 근거나 허용 조치가 없어 보류한다.

HYD-01-2edd SENSOR_CHECK ['get_recent_window', 'search_sop'] 온도 센서 품질 불량으로 설비 조치를 보류하고 센서 점검을 제안한다.

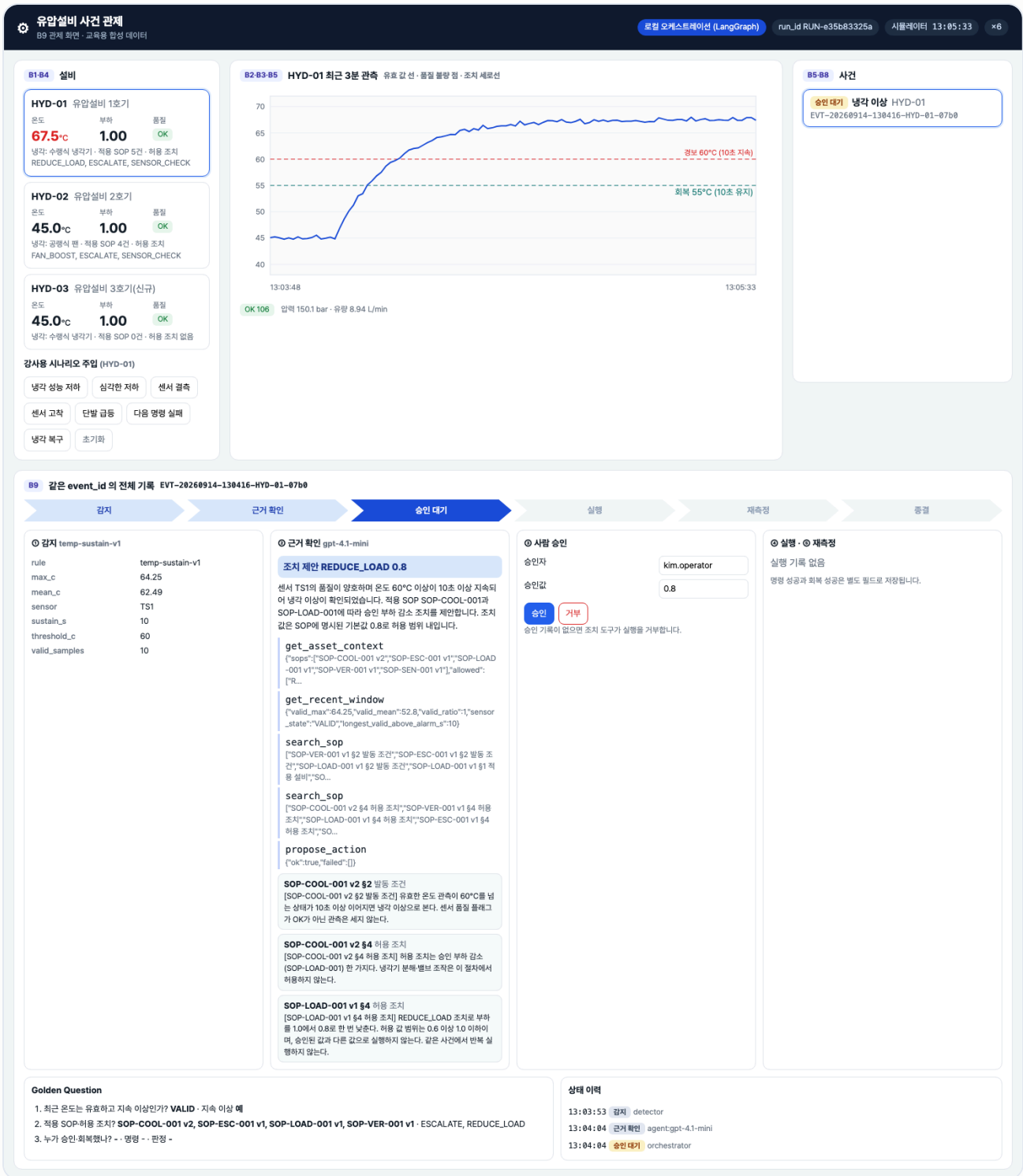
HYD-01-f81e ESCALATE ['get_recent_window', 'get_asset_context', 'search_sop', 'propose_action'] 허용 범위 검사 실패: ['event_open', 'not_already_executed']

- 센서 오류 사건은 `get_recent_window` 의 `sensor_state: SENSOR_FAULT` (유효 비율 0.75)를 보고 설비 맥락을 조회하지 않고 SOP-SEN-001 로 간다.
- 세 번째 사건은 이미 조치·재측정까지 끝난 CLOSED 사건이라 검사가 실패한다. 같은 사건을 반복 실행하지 않는 규칙(`not_already_executed`)이 에이전트 단계에서도 드러난다.



HYD-03 근거 없음 보류

같은 HYD-01 CLOSED 사건에 기록된 실제 LLM(gpt-4.1-mini) 제안은 `event.proposal` 에 남아 있다. 도구 순서는 `get_recent_window → get_asset_context → search_sop(query "허용 조치") → propose_action` 이었고, 인용은 `SOP-COOL-001 v2 §4` 와 `SOP-LOAD-001 v1 §4` , `citation_problems` 는 비어 있었다. 오프라인 에이전트(§2·§3 인용)와 인용한 절은 다르지만 둘 다 검색 결과에 있는 절이라 검증을 통과했다. 관제 화면에서는 아래처럼 도구 호출과 인용 본문이 함께 보인다.



냉각 이상 승인 대기 — 도구 호출과 인용

흔한 실수

- version 을 문자열과 정수로 섞어 비교한다('2' ≠ 2). 검증은 int(version) , str(section) 으로 맞춘다.
- 근거 검증을 LLM 모드에만 건다. 오프라인 에이전트도 코드를 바꾸면 틀린 인용을 만들 수 있다. propose_for_event 는 두 모드 모두 마지막에 검증한다.

- HOLD 를 실패로만 본다. HYD-03 의 HOLD 는 이 시스템이 바라는 결과다. 확인할 것은 "왜 멈췄는지"가 `rationale` 과 도구 기록으로 남았는가다.
- 도구 인자를 `list[dict]` 로 둔다. OpenAI 함수 스키마가 거부했던 실제 문제라 `Citation(doc_id, version, section)` 타입을 쓴다.

7. 실습 과제

- 실전반: `labs/practical/S05/`
 - `search_lab.py` `build_filter`: `|asset_id|` 경계 · `status active`
 - `search_lab.py` `confirm_with_graph`: `applicable_keys` 확인 · 같은 절 중복 제거
 - `search_lab.py` `verify_citations_lab`: `(doc_id, version, section)` 세 값 비교 · 인용 0개 보류
 - `skills/hydraulic-cooling-response/SKILL.md`: tools · 판단 순서 · 인용 규칙 · 중단 규칙
- 검증 방법: 필터 dict 와 경계 사례(`|HYD-011|`), 가짜 후보에 대한 그래프 확인, HYD-01-02-03 검색(폐기본·다른 사건 유형·다른 설비 없음), 지어낸 인용 HOLD, 오프라인 에이전트 네 경로, SKILL.md 정적 검사와 로더.
- 테스트는 `HYDOPS_AGENT_MODE=offline` 이며, 벡터 검색은 Neo4jVector 필터 의미를 따르는 오프라인 검색기(Neo4j SOPSection 읽기 전용)로 한다.
- 확인 명령: `cd lecture/system && PYTHONPATH=. .venv/bin/python -m pytest ../labs/practical/S05 -q`

시작본 실행 결과:

```
10 failed, 3 passed in 0.40s
```

모두 고친 뒤: `13 passed` .

8. 완료 확인 체크리스트

- ☐ 시험 구간 해시 `2bb1593b85bc83a9` , 그래프 SOP 7·SOPSection 42 를 확인해 환경 복구를 기록했다.
- ☐ 도구 4종의 입력 인자와 출력 요약을 한 줄씩 적었다(원시 배열이 없다는 점 포함).

- 시작본 검색에서 폐기본(SOP-COOL-001 v1)과 다른 사건 유형(SOP-SEN-001)이 섞이는 결과를 보았고, 고친 뒤 시스템 `search_sop` 와 같다.
- SKILL.md 의 tools 에 실행 도구가 없고, 판단 순서가 센서 품질부터 시작한다.
- 인용 규칙에 절 번호가 없고 "검색 결과에 실제로 나온 절만" 이 있다.
- 중단 규칙에 근거 없음 → HOLD, 센서 오류 → SENSOR_CHECK 가 있다.
- `SOP-COOL-001@v1#4` 인용이 HOLD 로 바뀌었다.
- HYD-01 REDUCE_LOAD · HYD-02 FAN_BOOST · HYD-03 HOLD · 센서 오류
SENSOR_CHECK 를 인용과 함께 확인했다.
- Skill(지침)과 SOP(근거)와 도구(검사)의 역할 차이를 한 문장씩 말할 수 있다.
- `pytest ../labs/practical/S05 -q` 가 13 passed 다.

9. 회고 (10분)

- 오프라인 에이전트는 SOP-COOL-001 v2 §2-§3 을, LLM 은 §4 를 인용했다. 둘 다 검증을 통과했다. 승인자에게 더 쓸모 있는 인용은 어느 쪽이고, 그 차이를 Skill 에 어떻게 적어야 절 번호를 박지 않고 유도할 수 있을까?
- 그래프 확인(검 ③)만 있으면 필터 ①②가 필요 없어 보인다. HYD-02 검색에서 `k×3` 후보가 모두 HYD-01 절이었다면 어떤 결과가 나올까?
- SKILL.md 에 "승인 없이 실행하지 않는다"를 적는 것과
`propose_action` · `execute_sim_action` 에 검사를 두는 것은 각각 무엇을 보장하고 무엇을 보장하지 못하는가?

개인 설명 과제 — 내가 설명할 질의 또는 실패 분기 하나: 팀원 각자 다음 중 하나를 골라 2분 안에 설명한다. (a) HYD-02 냉각 이상 검색에 SOP-SEN-001 이 섞이는 경로와 막는 줄 · (b) `SOP-COOL-001@v1#4` 인용이 HOLD 로 바뀌는 분기 · (c) 센서 오류 사건에서 `get_asset_context` 를 부르지 않는 분기 · (d) CLOSED 사건의 `propose_action` 이 `ESCALATE` 가 되는 분기.

강사 노트

시간 배분

시각	내용
14:00~14:50	활동 1 환경 복구 (컨테이너·해시·그래프 20분, 도구 호출 25분, 복구 실패 팀 지원 5분)

시각	내용
14:50~15:40	활동 2 검색 필터 (시작본 결과를 먼저 보이고 수정)
15:40~16:10	휴식 30분
16:10~17:00	활동 3 SKILL.md (절 번호 사고 사례 10분, 수정 30분, 로더 확인 10분)
17:00~17:50	활동 4 근거 검증·오프라인 경로·실제 사건 비교, pytest
17:50~18:00	회고

사전 준비

- 방학 중 컨테이너·볼륨 상태를 전달 확인한다. Neo4j의 SOPSection 42절과 벡터 인덱스 (`sop_section_openai`)가 있는지 `SHOW INDEXES` 로 본다. 리허설 환경에는 OpenAI 인덱스만 있고 오프라인 인덱스(`sop_section_offline`)는 없다. 키가 없는 교실에서 시스템 `search_sop` 를 시연하려면 강사가 `HYDOPS_AGENT_MODE=offline` 으로 `search.index_sections()` 를 수업 전에 한 번 실행한다(기존 OpenAI 임베딩은 그대로 두고 오프라인 속성·인덱스를 더한다). 학생 lab 테스트는 인덱스가 없어도 통과한다.
- 관제 서버는 `scripts/servers.sh local 4` (강사). 활동 4의 실제 사건 ID는 수업 DB의 `/api/events?all_runs=true` 에서 CLOSED·HOLD_NO_EVIDENCE·SENSOR_CHECK 하나씩 골라 칠판에 적는다.
- 교육용 플랫폼 bootstrap은 오늘 필요 없다. 6회차를 위해 `--upto datasource` 까지만 해 둔다.
- 11월 체크포인트 폴더가 팀별로 있는지 확인한다. 없는 팀은 `fixed_split()` 해시를 강사 기록과 비교한다.

장애 대응

- OpenAI 키·네트워크가 없으면 `HYDOPS_AGENT_MODE=offline` 으로 모든 활동을 진행한다. 활동 2의 `neo4j_retrieve` 는 오프라인 모드에서 SOPSection의 `embedding_offline` 속성을 읽으므로(필터가 있는 검색은 인덱스 대신 속성으로 코사인을 계산한다), 그 속성이 없으면 lab 테스트의 오프라인 검색기 결과로 대신 설명한다(점수·순위는 OpenAI 임베딩 결과와 다르다고 밝힌다).
- 체크포인트 복원이 필요하다면 강사가 팀의 `S04_code.tar.gz` 만 풀어 준다. `events.sql` 은 공유 DB에 되돌려 넣지 않는다(사건 중복 억제 인덱스와 충돌할 수 있다).
- `pytest tests` 를 누군가 실행해 사건 기록이 비었으면, 관제 화면 초기화 후 강사가 시나리오를 다시 주입한다.

용어

용어	뜻
Skill (SKILL.md)	업무 에이전트에 할당하는 판단 순서·인용 규칙·중단 규칙 문서. 근거가 아니라 지침이다
SOP 절	적용 설비·발동 조건·확인할 관측·허용 조치·승인 주체·재점검 기준 여섯 절로 된 조치 매뉴얼의 한 부분. 인용 단위
RAG	실행할 때 문서를 검색해 근거로 넣는 방식. 모델을 학습시키지 않는다
메타데이터 필터	벡터 검색 전에 <code>asset_scope · status</code> 같은 속성으로 후보를 거르는 조건
그래프 확인	검색된 절의 SOP 가 <code>(SOP)-[:APPLIES_T0]->(Asset)</code> 관계와 사건 유형을 실제로 갖는지 확인하는 단계
폐기본 (superseded)	새 버전이 나와 더 이상 적용하지 않는 SOP 버전. 감사용 검색에서만 포함
근거 검증	제안의 인용이 이번 실행의 검색 결과에 <code>(doc_id, version, section)</code> 그대로 있는지 코드로 확인하는 단계
HOLD	근거 없음·검사 실패로 조치를 제안하지 않고 멈춘 결정
SENSOR_CHECK	센서 품질 불량으로 설비 조치를 보류하고 센서 점검을 제안하는 결정
오프라인 에이전트	LLM 없이 Skill 순서를 코드로 따르는 결정적 에이전트 (<code>run_offline_agent</code>)